A Silent Hill 1 Style Town Level for Quake 3

Direct PBR, shadow mapping, and SH indirect lighting across NetRadiant-custom, q3map2 and ioquake3

Davis Wen

Revision 6 — 2 October 2026
(revision 5: 2 October 2026; revision 4: 1 October 2026; revision 3: 1 October 2026; revision 2: 17 September 2026; revision 1: 16 September 2026)

1 Goal

Build a playable Silent Hill 1 style town level — a fog-bound network of watertight streets under a low virtual sky, with enterable interiors — that loads in the customised ioquake3 as **ordinary Quake 3 content**, and extend the authoring and rendering pipeline until it looks the way the reference does.

The level is a Quake 3 map, not a new game. One engine binary, one gamepack, one BSP format, stock `scripts/*.shader` materials, and the editor's physical light entities. It is a **new** level, authored from the start with the enhancements in mind; enhancing existing Quake 3 levels is not part of this plan (§8.2).

The compiled level has three targets, in order of precedence:

1. **Authoring.** NetRadiant-custom under `Q3.game` creates and edits it, including its PBR materials as text, and q3map2 compiles it, with no other gamepack involved.
2. **Stock.** An unmodified ioquake3, with `opengl1` or its default `opengl2`, at default cvars runs the same pk3 as plain lightmapped Quake 3 content: a lightmap baked by `sh-baker` inside q3map2 (Stage 4), base textures, vanilla shader stages, fog volumes. It uses **no rend2 features** — no normal or specular maps, no `r_pbr`, no HDR lightmaps. This is what makes the level uploadable to `lvlworld.com`, and it should look as faithful to the enhanced target as a lightmap allows.
3. **Enhanced.** The customised ioquake3 with `renderer_sh` (the new sh-renderer) runs the same pk3 with direct PBR, shadow maps and SH indirect. PBR levels are developed for this target.

Original, non-PBR Quake 3 levels are unaffected: they keep running on `opengl2`. Every enhancement is ignored by the stock target. None replaces stock data. §6 turns this into rules every stage must satisfy.

2 What changed in revision 6

The compiler does not have to reproduce any original look: there are no original levels to rebuild, and what ships is the pk3. It must run on vanilla `opengl1`/`opengl2` and carry everything `renderer_sh` needs. Within that, the easiest path in the editor wins.

Lights: the editor's physical model, not q3map2's.
The light entities the editor already previews — `light` (point; `intensity` radiant flux, `radius` cutoff), `light_spot` (`intensity`, `radius`, `cone`, `cone_inner`, `target` or `angles`)

and `light_sun` (irradiance `intensity`, `angles`), with `_shadows` on spots and suns — are the one light model, shared by the editor preview, `sh-baker` and `renderer_sh`. Point lights never cast; one sun is honoured. Keys keep their names. There is no q3map2 light model to copy, no constant K and no calibration against q3map2 lightmaps. The editor does not read `q3map_sun`. §15.

`Q3.game` gets the PBR entity module (A6).

A tracked overlay in `setup/data/gamepacks/pbr/` installs `games/Q3.game` — the downloaded one with `entities="pbr"` — and `Q3.game/baseq3/_pbr_lights.ent`, defining `light`, `light_spot` and `light_sun` with the physical keys. Its name sorts before `entities.ent`, and the first entity class of a name is kept, so its `light` wins. It is installed with `sh install-gamepack.sh setup/data/gamepacks/pbr install/gamepacks` after the downloaded packs. A6 is folded into `q3-shader-pbr-materials`.

Vanilla compatibility of the light entities.

The stock game prints `light_spot doesn't have a spawn function` (and the same for `light_sun`) once per entity (`code/game/g_spawn.c:281`) and discards it; `light` is freed silently by `SP_light`. Harmless; the stock check allowlists the line. `-keeplights` keeps every light in the entity lump for `renderer_sh`. q3map2 treats any classname prefixed `light` as a light (`light.cpp:330`), so its placeholder lightmap uses its own reading of them.

`sh-baker` writes the vanilla lightmap.

Stage 4.3 always writes irradiance into the BSP lightmap pages, plus the light grid and vertex colours, from the same lights, besides the SH side file. This is no longer optional or conditional on 0.3. The irradiance-to-byte encoding is ours to choose so the result looks good on vanilla, for example a roll-off rather than a hard clip at the overbright limit. Until 4.3 lands, q3map2 `-light` produces a placeholder lightmap that is not expected to match the preview. 0.3 measures the `sh-baker` lightmap after 4.3.

q3map2 stays the host.

BSP, `-meta`, `-vis`, lightmap UV pages, light-grid points and vertex lists remain q3map2's; only its lighting maths (`-light`) is replaced.

Emissive strength has no Quake 3 scale.

No scale constant S and no `include/pbrunits.h`: emissive radiance is texel × `rgbGen const` colour × `qer_pbr_emissiveStrength`, added to the base pass in the light passes' linear units.

The editor's shadow-caster rule is authoritative.

Never cast: sky, fog, hint, trigger, areaportal, `noshadows`, player and bot clip, water, slime, lava, and `trans` without `alphashadow`. Solid caulk casts; alpha-tested shaders cast through their preview alpha test; other drawn opaque surfaces cast. `sh-baker` follows the editor; nothing claims to mirror q3map2.

`pbr.game` stays frozen; its light entities move.

A7 still deletes `pbr.game` and the `.mtr` parser but keeps the `pbr` entity module, which `Q3.game` now uses.

Work tracking.

`q3-stock-lights` is deleted. The changes are `q3-shader-pbr-materials` (A1–A3, A6) and `q3-bench-and-stock-check` (A4, A5). The benchmark keeps its lights as they are; only `bench.mtr` becomes `scripts/bench.shader`, with no `q3map_sun` conversion and no brightness re-tune in the stock engine (brightness is tuned in the editor preview). The stock check drops calibration mode. It still compiles with `-meta -keeplights`, `-vis -saveprt`, `-light -patchshadows`; the last gives the placeholder lightmap.

Engine parity.

3.4 is parity with the editor's physical light model. The `-shbake` stage learns `qer_pbr_*` and the editor's caster rule.

3 What changed in revision 5

Kept for the record. Revision 6 supersedes its light decisions — q3map2's model, the constant K and its calibration, `q3map_sun` suns, `_sh_shadows` and the emissive scale S — along with the stock-engine brightness tuning of the benchmark, the optional lightmap write and the `q3-stock-lights` change (§2).

PBR levels target sh-renderer only; rend2's PBR features are not used.
rend2 is built around the lightmap and cannot run a BRDF over metallic–roughness, occlusion and normal maps the way sh-renderer does. Revisions 3 and 4 shipped data for it anyway (a `normalMap` stage, a generated `_s` pack, `lm_*.hdr` calibrated to rend2). That is withdrawn. PBR data lives only in top-level `qer_pbr_*` keywords, including the normal map (`qer_pbr_normal`), and shader stages stay vanilla. Original non-PBR levels fall back to `opengl2` as before.

The stock target is the lvlworld fallback, and `opengl1` is back in.
With no rend2 keywords in shader text, `renderergl1` no longer rejects the shaders, so the stock check runs both `opengl1` and `opengl2` at default cvars. Its fidelity comes from q3map2's lightmaps; §0.3 measures how close it gets.

One pk3.
The base/overlay split and the rend2-PBR pack are dropped. One pk3 carries the BSP, shaders, base textures, lightmaps, PBR maps and, later, the SH side file. Stock renderers never reference the PBR files.

The normal-map syntax of revision 4 was wrong anyway.
rend2 has no `normalMap <image>` keyword: a normal map is a stage containing `stage normalMap` and `map <image>`, and the bare form makes rend2 reject the shader (`renderergl2/tr_shader.cpp:870`, `:1267`).

Lights: q3map2's model, exactly, scaled by one constant.
The editor preview reproduces q3map2's falloff, spot cone and suns and multiplies by one constant K derived from the lightmap path; `renderer_sh` adopts the same model (3.4). q3map2's `light`×7500 is a radiant *intensity*, not a flux. Point lights do not cast shadows in the preview. The per-light shadow switch is `_sh_shadows`. §15.

Compile flags.
`-keeplights` is a BSP-stage option (`bsp.cpp:658`), not a light option; `-patchshadows` is added to `-light` because q3map2's Quake 3 default leaves patches unshadowed while the preview's patches cast.

Work tracking.
Tasks A1–A6 are specified as three openspec changes: `q3-shader-pbr-materials` (A1–A3), `q3-bench-and-stock-check` (A4–A5) and `q3-stock-lights` (A6).

4 What changed in revision 4

Kept for the record. Revision 5 supersedes its use of rend2's `normalMap` stage, the overlay pk3 and `lm_*.hdr` (§3); revision 6 supersedes its handing of q3map2's parsed lights to the baker (§2).

The export pipeline was a prior attempt, not the pipeline.
`ioq3-map-exporter` and `ioq3-materialize` were built to export existing Quake 3 levels and enhance them with no editor in the loop. That did not work well: those levels are inherently incompatible with the modern pipeline. Revisions 2 and 3 treated both tools as live consumers and drew conclusions from them — that `.mtr` "has no consumer", that PBR maps would be generated and found by file-name suffix, that the exporter's light scale was part of the chain. Those conclusions are withdrawn. The move onto Quake 3

shader conventions stands, for a different and stronger reason: the stock target cannot load anything else.

PBR materials are authored as text in `.shader` files.
As original NetRadiant does for Quake 3 shaders, with no material GUI. Roughness, metallic, occlusion and their factors are top-level `qer_pbr_*` keywords; the normal map is rend2's own `normalMap` stage; base colour and emissive are ordinary stages. §7.

The prefix is `qer_pbr_`.
"qer" is QuakeEd Radiant, the namespace for editor hints. Both ioquake3 renderers skip any top-level keyword starting with `qer` (`renderergl2/tr_shader.cpp:1718`), and q3map2 ignores unknown `qer_` keywords while it warns on unknown `q3map_` ones (`shaders.cpp`, "Unknown q3map_* directive"). The keywords are read by the editor, q3map2's SH stage and `renderer_sh`; the prefix is chosen because stock engines skip it.

The suffix convention of revision 3 is withdrawn.
Maps are named explicitly in shader text. What remains is a naming hazard: stock rend2 auto-loads `<image>_n`, `_nh` and `_s` siblings even without a keyword, so those suffixes are reserved (§7).

SH baking moves into q3map2.
The prior pipeline baked SH into a single xatlas atlas over an exported glTF and rendered that glTF; nothing ever mapped back to the BSP. Revision 4 links `sh-baker` into q3map2 as a library and bakes into q3map2's own lightmap pages, through a new `-shbake` stage. `sh-baker` gains a library API that serves both xatlas atlases and multi-page layouts, delivered as reviewed pull requests to its repository. Stage 4 is rewritten.

The exporter's light conversion leaves the chain.
q3map2 hands its own parsed lights to the baker, so the q3map2 interpretation reaches the bake directly. The units section drops the exporter row.

Open: parser reuse for `renderer_sh`.
Revision 2 planned for `renderer_sh` to consume the exporter's parsing code (`libioq3_map`). With the exporter out of the pipeline this is a decision, not a given. §6.2.

5 Earlier revisions

Revision 3 made the three targets explicit; dropped `rendererg11`, which rejects a whole shader on an unknown keyword (`rendererg11/tr_shader.cpp:972`, `:1588`, default shader at `:2491`); froze `pbr.game` and made `Q3.game` the only authoring target; found that a PBR specular map is not stock-safe; made q3map2's light interpretation authoritative (withdrawn by revision 6); required a complete BSP lightmap; recognised that stock rend2 loads `lm_*.hdr`; added the base/overlay packaging and the stock-engine check. Its material-suffix convention was withdrawn by revision 4; dropping `rendererg11`, shipping `lm_*.hdr` and the base/overlay packaging are withdrawn by revision 5 (§3). The rest stands.

Revision 2 reframed the target as a Quake 3 level rather than a new game; moved lights onto the stock `light/target/q3map_sun` convention (withdrawn by revision 6); found that `-keeplights` already exists (`bsp.cpp:658`, persisted as `_keepLights` at `writebsp.cpp:230`); measured what rend2 already ships; decided that `sh-renderer` becomes `renderer_sh` (§3.1); and re-ordered the work so that fog, darkness and the flashlight come before SH indirect. Its rationale for retiring `.mtr` rested on the export pipeline and is replaced by revision 4's.

Retained from the completed editor work (stage1 and stage2 changes, already archived): the GLSL PBR programs and the HDR/tonemap resolve path, the whole shadow-map subsystem (cascades, spot atlas, bias scheme, PCF, `radiant/cascade.h`), the `IShader` PBR accessors (`include/ishaders.h:137-139`), light plumbing in the entity plugin, the benchmark map, and

the units and shadow-constant documentation. **Retargeted:** how a material is *identified*, not how it is shaded, and which game the light entities are authored under.

6 Compatibility contract

Rules every stage satisfies. "Stock" means ioquake3 with `cl_renderer opengl1` or `opengl2` and default cvars.

Content	Rule	Grounds
Shader text	Vanilla Quake 3 shader keywords only, plus top-level `qer_pbr_*`. No rend2 keywords (`stage`, `specularReflectance`, `gloss`, `roughness`, `normalScale`, ...) and no other custom keywords.	An unknown keyword invalidates the whole shader in both renderers (`renderergl2/tr_shader.cpp:1267`, `renderergl1/tr_shader.cpp:972`); top-level `qer*`/`q3map*` are skipped by both, but not inside stages.
Materials	As §7.	
rend2 siblings	No image named `<base>_n`, `_nh` or `_s` next to a stage image.	rend2 loads those with no keyword (`renderergl2/tr_shader.cpp:2150-2215`) and would apply PBR data through its lightmap path.
Lights	`light`, `light_spot`, `light_sun` with the editor's physical keys (`intensity`, `radius`, `cone`, `cone_inner`, `_color`, `target`/`angles`, `_shadows`), kept by `-keeplights`.	§15. The stock game never uses light entities.
Entities	Classnames the game code spawns, plus `light_spot` and `light_sun`. Other enhanced-only parameters are extra keys.	Unknown keys are ignored. An unknown classname is discarded after one `doesn't have a spawn function` line (`g_spawn.c:281`), which the stock check allowlists for those two.
Fog	Stock fog volumes (`fogparms`).	Works on every target; `renderer_sh` implements the same semantics.
Lightmaps	The BSP lightmap lump is always complete and is the stock target's only lighting; it is baked by `-shbake` (4.3), with q3map2 `-light` as the placeholder until then. No `lm_%04d.hdr` and no deluxe maps are shipped.	rend2 would load `maps/<name>/lm_%04d.hdr` at `r_hdr 1` (`tr_bsp.cpp:279-326`) and apply it in place of the lump.

Packaging. One pk3 per level, as uploaded to `lvlworld.com`: the BSP, the levelshot (`levelshots/<map>.jpg`; without one the engine reports a missing image), `scripts/*.shader`, base-colour and emissive images, the lightmaps, the `qer_pbr_*` maps (normal, metallic–roughness, occlusion) and, from Stage 4, the SH side file. Stock renderers never reference the PBR files; `renderer_sh` reads all of it. The cost is download size.

6.1 Verification

The stock check (`tools/stockcheck/`) runs the fork's ioquake3 build (`ioq3-custom/ioq3/build/Release`). Its renderers were machine-converted to C++ and are otherwise original; each run records the fork's commit so drift is traceable. Matrix: `cl_renderer opengl1` and `opengl2`, default cvars. Each run uses a clean temporary basepath and homepath holding only the stock paks and the pk3, `+set developer 1 +set logfile 2 +devmap <map>`, then for each documented view (`maps/<map>.cameras`) `setviewpos`, a settle wait, `viewpos` (the reported eye position is recorded in `views.txt`, because `setviewpos` pushes the player forward) and `screenshot`. It fails on any `WARNING` line or `<classname> doesn't have a spawn function` line not on a reviewed allowlist (which starts with the `light_spot`/`light_sun` spawn-function lines), on `R_FindImageFile could not find`, and on `Couldn't find image file for shader` (`developer` only, `tr_shader.cpp:3336`). A missing image is excused only by an allowlist entry naming the exact line of an image the engine replaces with a built-in fallback (`rendererg12`'s `gfx/2d/sunflare`). It also fails on a missing screenshot, on a `viewpos` that is missing or more than 256 units from its request (an ignored teleport), on an abnormal exit and on a timeout. The compile (`-meta -keeplights`, `-vis -saveprt`, `-light -patchshadows`) fails on a leak or on q3map2's `Unknown q3map_* directive`. The engine only parses shaders a map uses, so the check covers exactly the shaders the map references. The enhanced check is the same run with `cl_renderer sh`. There is no calibration mode: nothing is matched against q3map2's lightmap.

6.2 Open decisions

- **Game code.** baseq3 needs nothing built but has no flashlight on any target. A small mod's cgame can attach a light to the player with `trap_R_AddLightToScene` on every engine, stock included; that call is omni-only (`tr_public.h:75`: origin, intensity, colour), so a spot flashlight needs a `refexport_t` extension that only the customised engine has, with the omni light as the stock fallback. The BSP and pk3 rules do not depend on the choice.
- **BSP and shader parsing in `renderer_sh`.** Reuse the exporter's parsing code (`libioq3_map`: BSP lumps, patch triangulation, shader scripts, entities), or write the runtime's own. The first is less code; the second avoids carrying a library whose parent tool is out of the pipeline. q3map2's SH stage does not depend on this: it uses q3map2's own parsing.

7 Material authoring

A PBR material is an ordinary lightmapped Quake 3 shader, written in `scripts/*.shader` in the editor's Shader Editor or any text editor. Its stages are vanilla; all PBR data is in top-level `qer_pbr_*` keywords, which every stock renderer skips:

```
textures/town/brick
{
    qer_editorimage textures/town/brick_c
    qer_pbr_normal textures/town/brick_nrm             // tangent space, linear
    qer_pbr_metallicRoughness textures/town/brick_mr   // G = roughness, B = metallic
    qer_pbr_occlusion textures/town/brick_ao           // R = occlusion
    qer_pbr_roughnessFactor 0.8
    q3map_lightmapSampleSize 8
    {
```

```
        map $lightmap
        rgbGen identity
    }
    {
        map textures/town/brick_c          // base colour
        blendFunc GL_DST_COLOR GL_ZERO
        rgbGen identity
    }
}
```

Property	Written as	`.mtr` equivalent
Base colour (sRGB)	The image of the stage the lightmap multiplies. Stages are written explicitly.	`basecolor`
Normal (linear)	`qer_pbr_normal <image>`: tangent space, OpenGL Y-up.	`normal`
Metallic–roughness (linear)	`qer_pbr_metallicRoughness <image>`: G roughness, B metallic. Without it, metallic is `qer_pbr_metallicFactor` (default 0) and roughness is `qer_pbr_roughnessFactor` (default 1).	`metallicroughness`
Occlusion (linear)	`qer_pbr_occlusion <image>`: R occlusion.	`occlusion`
Factors	`qer_pbr_roughnessFactor <f>`, `qer_pbr_metallicFactor <f>`, `qer_pbr_baseColorFactor <r> <g> <b>`. With a metallic–roughness map they default to 1 and multiply it, as in glTF.	`*factor`
Emissive (sRGB)	An additive stage (`blendFunc add`) carrying the emission image, optional `rgbGen const`. `qer_pbr_emissiveStrength <f>` (default 1) scales it: radiance = texel × colour × strength, in the light passes' linear units with no Quake 3 specific scale. Stock renderers draw the stage unscaled.	`emissive`

- **Why emissive is a stage.** An additive stage is how Quake 3 already draws glow, so the stock target shows it unchanged; `sh-baker` already bakes additive layers as area lights ("Bake additive SH_material_layers surfaces as area lights", `843237b`), so the glow also reaches the lightmap once 4.3 lands; `q3map_surfacelight` is not needed.
- **Factors are enhanced-only.** The stock target never sees `qer_pbr_baseColorFactor`; where the stock look matters, bake the factor into the texture instead.
- **Reserved suffixes.** Never name an image `<base>_n`, `<base>_nh` or `<base>_s`, where `<base>` is a stage image: rend2 picks those up with no keyword (`renderergl2/tr_shader.cpp:2150-2215`). The example above uses `_nrm`, `_mr` and `_ao`.
- **No rend2 stage keywords.** `stage normalMap`, `stage specularMap`, `roughness` and the rest are rend2-only and make `renderergl1` reject the shader.
- **Non-PBR shaders.** A lightmapped opaque shader without `qer_pbr_*` keywords is

shaded in the editor preview (and in `renderer_sh`) as a fully rough dielectric with its base colour, and `sh-baker` bakes it the same way.

- **Editor support** is Stage 2.2: the Quake 3 shader parser reads all of the above and fills the existing `IShader` PBR accessors, the Shader Editor completes the new keywords, and the lighting preview renders the result.

8 Current state (verified)

8.1 The engine: ioquake3, renderer modules now C++

- The renderer is a runtime-loaded module: `code/client/cl_main.c:3184` registers `cl_renderer` (default `opengl2`, `CVAR_ARCHIVE|CVAR_LATCH`), builds `renderer_<name>` and resolves `GetRefAPI` (`:3204`). `rendergl1` and `rendergl2` ship side by side; a new backend is a sibling, not a replacement.
- **Both renderer modules are now C++** (`0135354f`, "CPP Renderers"), and `tr_public.h` already carries `extern "C"` guards (`:25`, `:214`). The `refexport_t`/`refimport_t` boundary is therefore usable from C++ directly. **This obsoletes the separate C-ABI wrapper** previously planned for `sh-renderer` (`ioq3-custom/docs/c_interface_design.tex`); only the control inversion — initialise against an existing GL context, expose discrete load/update/render/resize/shutdown calls, point the GL loader at the engine's — remains necessary.
- `rendergl2` (rend2) already implements, off or on by cvar in `tr_init.cpp`: `r_pbr` (`:1241`, default **0**), `r_hdr` (`:1201`, default 1), `r_toneMap` (`:1206`), `r_normalMapping` and `r_specularMapping` (`:1225-1227`, both default 1), `r_deluxeMapping` (`:1229`), `r_cubeMapping` (`:1236`), `r_sunlightMode` (`:1271`, cascaded sun shadows), `r_dlightMode` (`:1252`; ≥ 2 gives per-pixel dynamic lights *with* projected shadows), plus SSAO and a tonemap post chain.
- Under `USE_PBR`, `glsl/lightall_fp.glsl:398-404` is metallic–roughness with $F_0 = \mathrm{mix}(0.04, \mathrm{basecolor}, \mathrm{metallic})$, Disney diffuse and GGX-ish specular. Maps come from the stage keywords `normalMap` (`tr_shader.cpp:881`) and `specularMap` (`:894`), the `roughness` keyword (`:976`), or the `_n`/`_nh`/`_s` siblings it loads without a keyword (`:2150-2215`).
- **Specular map interpretation depends on client cvars.** With `r_pbr 0` (default) it is RGB reflectance plus alpha gloss; with `r_pbr 1`, red is gloss under `r_glossType` (default 1, smoothness) and green is metallic. Colour space differs too: `lightall_fp.glsl:394` uses `diffuse.rgb *= diffuse.rgb`, a gamma-2.0 approximation, where the editor uses the exact piecewise sRGB curve.
- **rend2 loads external HDR lightmaps**: `maps/<name>/lm_%04d.hdr`, promoting the lightmap atlas to `GL_RGBA16` when the first one exists and `r_hdr` is on (`tr_bsp.cpp:279-326`).
- **None of rend2's PBR, specular, normal-map or HDR-lightmap features are used** (revision 5). They are listed here because the contract must keep rend2 from picking them up by accident.

8.2 The prior attempt

`ioq3-map-exporter` (`.pk3` → IBSP → glTF), `ioq3-materialize` (Real-ESRGAN upscale, DeepBump normals, ORM packing), `sh-baker` and `sh-renderer` were built to enhance existing levels such as `q3dm1` and `mpteam3`. How the bake was wired, which is why it does not carry over as-is:

- The exporter writes each vertex's q3map2 lightmap coordinates, which are local to one lightmap page, as `TEXCOORD_1`.
- `sh-baker` ignores them: its loader reads only `TEXCOORD_0` (`src/loader.cpp:170`, `:212`). It builds a fresh atlas with xatlas, splitting vertices along chart seams, and always builds a single atlas, whose size xatlas picks near the requested width (`src/atlas.cpp:110-207`); `-skip_parameterization` still repacks the charts. It writes a new glTF with its own `TEXCOORD_1` (`src/saver.cpp:769`).
- `sh-renderer` renders that glTF (`src/loader.cpp:142`, `:941`), never the BSP.

Carried over: `sh-renderer` as `renderer_sh` (Stage 3) and `sh-baker` as a library linked into q3map2 (Stage 4). `ioq3-materialize` and the exporter's glTF output are not part of this plan; the exporter's parsing code is an open decision (§6.2).

8.3 The editor and q3map2

- Gamepacks are data; the lighting draw mode, the half-float target and tonemap resolve, and sun-cascade plus spot-atlas shadow maps are implemented and documented (`radiant/renderstate.cpp`, `radiant/cascade.h`, `setup/data/gamepacks/pbr/README.md`).
- The gating on "is this a PBR game" is a single string test, `renderstate.cpp:2202`, against the `.game shaders` key; per-material dispatch already exists (`renderstate.cpp:3054`, `m_shader->isPBR()`), so a mixed material set already renders correctly.
- Text authoring of PBR materials exists today only as `.mtr` under `pbr.game` (`ShaderTemplate::parsePBR`, `plugins/shaders/shaders.cpp:1498`), which the stock target cannot load. The Shader Editor (`radiant/gtkdlgs.cpp:599`) highlights and completes Quake 3 and q3map2 keywords; its table has none of rend2's.
- `Q3.game` already authors and compiles stock maps; it lacks the PBR keywords and the lighting preview.
- q3map2 assigns lightmap coordinates and pages during `-light` (`SetupSurfaceLightmaps`, `light.cpp:2964`); they are final after `StoreSurfaceLightmaps` (`lightmaps_ydnar.cpp:2253`). It has no `.hdr` writer. It builds with `-std=c++20 -fno-exceptions -fno-rtti` from the top-level `Makefile` (`:138`) and is GPL-2.0-or-later (`q3map2.h:9-11`).

9 Shortest path to a walkable town

Ranked by effect on the goal per unit of work. Items 1–3 need no new subsystem.

	Item	Why it comes first
1	Author the town under `Q3.game` and pass the stock check (Stage 0)	Nothing blocks it. Until the map exists, no rendering change can be judged, and every exit criterion below refers to it.
2	Stand up the stock check (Stage 0.4)	A small script. Keeps the pk3 lvlworld-safe from the first map; how faithful the lvlworld build looks is measured once Stage 4 bakes its lightmap (0.3).
3	Fog, darkness, carried flashlight (Stage 1)	This is what reads as Silent Hill. Mostly content, cvars and one carried dynamic light.
4	PBR materials and preview in `Q3.game` (Stage 2)	Text authoring of the town's materials, previewed in the editor. Its first step is additive and can land at once.
5	`renderer_sh` and engine parity (Stage 3)	The large build-out. Deliverable in tiers, because `opengl2` stays available throughout.
6	SH indirect baked by q3map2 (Stage 4)	Largest piece; makes interiors and fog-lit ambience read without hand-placed fill, but is not what defines the reference look. The `sh-baker` changes can proceed in parallel from now.

10 Stage 0: The town, with today's tools

Outcome. A walkable street block, authored under `Q3.game`, compiled by the stock pipeline, passing the stock check, with the town's visibility conventions already in force.

10.1 0.1 Convert the benchmark map

- `pbr.game/base/maps/bench.map` is already a street block with two enterable interiors, a low sky ceiling, a fog volume, a sun and spot lights. Copy it into the `benchmark` mod directory (`content/benchmark/`), rewrite `bench.mtr` as `scripts/bench.shader` in the form of §7 and rename the `_n` images. Its `light`, `light_spot` and `light_sun` entities stay as they are (revision 6); brightness is tuned in the editor preview, not in the stock engine.
- It becomes the parity and regression asset for every later stage, under the Quake 3 gamepack. It is the only map that needs this rewrite, because no further content is authored in `pbr.game`.

10.2 0.2 First town block

- Author under `Q3.game` and the conventions in the world-building section: low sky ceiling, structural shells, detail ornament, fog colour matched to sky, hint brushes across streets, area portals in doorways. Materials follow §7 from the first shader.
- Compile `q3map2 -meta -keeplights`, `-vis -saveprt`, `-light -patchshadows`; leak-free is the gate. `-keeplights` is a BSP-stage option that keeps the light entities in the entity lump for the real-time direct path. `-light` gives a placeholder lightmap until 4.3, when `-light -shbake` replaces its lighting with `sh-baker`'s. No `-deluxe`: deluxe maps are a

rend2 feature.
- Exit criterion: walkable in ioquake3 at playable framerate, PVS behaving like an indoor level, and the stock check of 0.4 passes.

10.3 0.3 Measure the stock fallback

- After 4.3, run the block on `opengl1` and `opengl2` at stock defaults — target 2, the lvlworld build, with `sh-baker`'s lightmap — and capture the documented views beside the editor's lighting preview and `renderer_sh`.
- **Deliverable: a written verdict** on how faithful the lightmapped fallback is and what it misses (normal-map detail, specular). It judges the lightmap encoding of 4.3 and may send it back for tuning. Before 4.3 the placeholder lightmap is only checked for loading and warnings.

10.4 0.4 Stand up the stock check

- Script the matrix of §6.1 in `tools/stockcheck/`. Run it on the benchmark map and the town block; add the enhanced run once `renderer_sh` reaches Tier 1.
- Exit criterion: both maps pass on `opengl1` and `opengl2`. From here on every stage's exit criterion includes this check.

11 Stage 1: The Silent Hill look

Outcome. The block reads as the reference: near-zero visibility, darkness with a moving pool of light, no discernible skyline.

- **Fog is the primary aesthetic, not a VIS trick.** Tune fog volume density and colour against the sky, with the far plane at fog opacity distance. Decide in `renderer_sh` whether fog composites before or after the tonemap — the reference look depends on which — and keep the stock fallback's `fogparms` reading close to it.
- **Carried flashlight.** A player-attached dynamic spot light, shadow-casting, is the defining mechanic. It needs the game-code decision of §6.2. In `renderer_sh` it is a shadowed spot using the editor's spot atlas scheme; the stock fallback gets the omni dynamic light that `trap_R_AddLightToScene` provides.
- **Grass and trees.** Overgrown verges and bare trees belong to the town's look. Build them as the benchmark does: `misc_model` meshes merged into the BSP by q3map2, every vegetation shader lightmapped through `q3map_forceMeta`, leaf cards alpha-tested and casting through their alpha, grass as static crossed quads that cast nothing. Scattering at town scale (by hand, or a compile-time scatter the editor cannot preview) is decided here.
- **Darkness budget.** With physical units and a tonemap, the ambient floor is what separates "dark and readable" from "black". Fix an exposure and ambient for the town and record them next to the map, as the benchmark already does.
- Exit criterion: a side-by-side against Silent Hill 1 reference stills that the author accepts. This is a judgement call, deliberately, and it gates nothing downstream.

12 Stage 2: Retarget the editor onto Quake 3 conventions

Outcome. One gamepack. PBR materials are authored as text under `Q3.game`, and the lighting preview predicts the engine because both read the same data the same way.

12.1 2.1 Decision: `pbr.game` is frozen

Decided (revision 3). No new content is authored in `pbr.game`; `Q3.game` is the only authoring target. A map authored in `pbr.game` uses `.mtr` materials, which the stock target cannot load, so it violates target 1 directly. Its light entities are not the problem: revision 6 moves them to `Q3.game` (2.2 step 4). Authoring geometry and shader text in `Q3.game` needs no editor change; the PBR keywords, the lighting preview and the light entities arrive with 2.2.

The retarget is code work, not a `.game` edit. Module selection at `radiant/plugin.cpp:196-199` takes exactly one module per key, and neither assignment gives both stock shaders and the preview:

- `shaders="quake3"` clears `g_pbrGame` (`renderstate.cpp:2202`), and with it the PBR shading path (`:3054`) and both shadow atlases — `renderstate.cpp:2165` returns early before building them.
- `shaders="pbr"` points the loader at `materials/*.mtr` with `g_enableDefaultShaders=false`, so stock `scripts/*.shader` files stop loading and bare textures get no fallback shader.

12.2 2.2 The retarget, in landing order

1. **Parse the material keywords** (additive; lands first). Teach the Quake 3 shader parser in `plugins/shaders/shaders.cpp` the `qer_pbr_*` keywords and emissive from the additive stage, and fill the existing `IShader` PBR accessors from them; the base colour is the image of the stage the lightmap multiplies. `isPBR()` becomes "declares any `qer_pbr_` keyword". rend2 stage keywords are not read (revision 5). *Done (`q3-shader-pbr-materials`).*
2. **Shader Editor completion.** Add the `qer_pbr_*` keywords to the keyword table in `radiant/gtkdlgs.cpp`, with argument hints. *Done (`q3-shader-pbr-materials`).*
3. **Lighting mode under `Q3.game`.** Replace the `renderstate.cpp:2202` game-identity test with a preference. With steps 1–3 the town's materials preview in the editor. *Done (`q3-shader-pbr-materials`).*
4. **Light entities under `Q3.game`** (revision 6; part of the first openspec change). Install the overlay of §2: `games/Q3.game` with `entities="pbr"` and `Q3.game/baseq3/_pbr_lights.ent`. The existing `LIGHTTYPE_PBR` path then previews `light`, `light_spot` and `light_sun` under `Q3.game`, with no change to the light code. Point lights do not cast shadows. *Done (`q3-shader-pbr-materials`).*
5. **Fold and delete.** Move the surviving gamepack content — build menu entries (including `-shbake` once Stage 4 lands), the benchmark map, the shadow and units documentation — into the Quake 3 pack; retire `SHADERLANGUAGE_PBR` and the `.mtr` parser; delete `pbr.game`. The `pbr` entity module stays, because `Q3.game` uses it. The build menu entry is the overlay menu's "Town" build (`-meta -keeplights`, `-vis -saveprt`, `-light -patchshadows`); `-shbake` joins it in 4.3. The documentation is `setup/data/gamepacks/pbr/README.md`; the benchmark is `content/benchmark/`. *Done (`retire-pbr-game`).*

Exit criterion: the town block renders in the editor's lighting mode under `Q3.game`, with screenshots from the documented views recorded. Numerical parity with `renderer_sh` is 3.4's criterion.

13 Stage 3: `renderer_sh` and engine-side parity

Outcome. A second renderer module that carries the project, then direct PBR and shadows matching the editor preview.

13.1 3.1 Decision: build the Quake 3 feature set out in sh-renderer

Decided. The missing feature set is added to `sh-renderer`, which becomes `renderer_sh`. `renderergl2` is left untouched. Rejected: modifying rend2 to host sh-renderer's SH reconstruction and Forward+ culling.

Rationale:

- **The work is additive.** Nearly every item in the inventory below is a new module in a codebase whose architecture is already written down and enforced. Hosting SH in rend2 would instead mean editing its shader-permutation system, its backend command buffer and its `tess` batching — where its accumulated complexity lives, and what is hardest to reason about from outside.
- **The two renderers coexist, so an incomplete one costs nothing.** `cl_renderer` is a latched cvar: `renderer_sh` can lack sky, marks and cinematics and still be useful for walking the town and judging light, with `opengl2` one cvar away as an A/B reference on the same map. Modifying rend2 has the opposite property — a half-finished integration degrades a renderer that works today, and that renderer is the stock target.
- **The architectural directives survive.** GL 4.6, DSA-only, Forward+, SSBOs stay intact rather than being recorded as superseded.

What changes in sh-renderer's inputs. It renders the BSP, not a glTF: draw surfaces with their q3map2 lightmap coordinates, materials from the shader text of §7, the SH bands from the side file of Stage 4. Its glTF loader stays for its own tests and tools. The parser it uses is the open decision of §6.2.

The one part that is not additive, and is planned as a refactor rather than a feature: `sh-renderer` currently owns its process. It uses `glog`, `gflags`, its own file I/O, and its own GLFW window and frame loop. As a renderer module it must route logging through `ri.Printf`/`ri.Error`, configuration through `ri.Cvar_Get`, *all* file access through `ri.FS_ReadFile` so assets come from `.pk3` via the engine's VFS, allocation through `ri.Hunk_Alloc`, and must initialise against an already-current GL context with glad pointed at the engine's loader. This touches every module's edges. Do it first, before any feature work, so no feature is ported twice.

13.2 3.2 Free: link, do not reimplement

`code/rendercommon/` is format and utility code rather than renderer architecture, and is already compiled into both existing modules: image decoders for TGA, JPG, PNG, BMP, PCX and PVR, plus `tr_font.c`, `tr_noise.c`, `tr_subs.c` and `puff.c`. Link it. `tr_public.h` and `tr_types.h` define the ABI and are shared by definition.

13.3 3.3 Feature inventory

`refexport_t` has 37 entry points. Ordered below by whether the town is playable without the item. The LOC figures are the corresponding rend2 sources, as an order-of-magnitude scope reference — not as code to copy.

Tier 1 — required to boot and walk the map.

- Module skeleton: `GetRefAPI`, the `refexport_t` table, `BeginFrame`/`EndFrame`/`Shutdown`, and the control inversion off GLFW.
- The `ri.*` refactor of §3.1.
- `LoadWorld` from `.bsp`, plus `SetWorldVisData`, `inPVS` and PVS-based cluster culling, and `GetEntityToken` (cgame reads the entity lump through the renderer).

- Materials from shader text: the base-colour stage, the `qer_pbr_*` keywords, additive emissive stages, and the defaults of §7 where absent.
- 2D: `SetColor`, `DrawStretchPic`, `RegisterShaderNoMip`. Without these there is no HUD, no menu and no console — an ortho quad batcher, small but unavoidable.
- Frame protocol: `ClearScene`, `AddRefEntityToScene`, `RenderScene`, `BeginRegistration`/`EndRegistration`, `LightForPoint`, `AddLightToScene`.
- MD3: `RegisterModel`, `RegisterSkin`, `ModelBounds`, `LerpTag`, and vertex-morph interpolation between frames — players, weapons and items, with `LerpTag` attaching weapons to hands. (`tr_model` 1378 + `tr_mesh` 407 LOC.)

Tier 2 — playable, but visibly wrong without it.

- **Sky** (`tr_sky`, 859 LOC). Highest priority of this tier for this project specifically: the low sky ceiling is the town's defining structural element.
- **Fog volumes**, with the stock `fogparms` semantics so the same map reads the same on both targets. The primary aesthetic of Stage 1; the town does not read without them.
- Multi-stage shader runtime: `tcMod`, `rgbGen`, blend funcs, `animMap`, `deformVertexes` (`tr_shade_calc`, 825 LOC, plus part of `tr_shader`, 3898). Already designed in `ioq3-custom/docs/layer_material_compositing.tex`.
- `MarkFragments` — bullet holes and blood decals (`tr_marks`, 467 LOC).
- `AddPolyToScene` — cgame's particle and effect path leans on it heavily.

Tier 3 — stub or defer.

- `DrawStretchRaw`, `UploadCinematic`, `TakeVideoFrame` — video playback and capture.
- `RegisterFont`, `RemapShader`, `AddAdditiveLightToScene`.
- Skeletal models, MDR and IQM (`tr_model_iqm`, 1850 LOC). `baseq3` does not use them; needed only if the project's own characters are skeletal, which is undesigned.
- Flares (`tr_flares`, 575 LOC), portals and mirrors.

Exit criterion for Stage 3.3: the town block is walkable under `cl_renderer sh` with a working HUD and player model, screenshots from the documented views can be compared against the stock fallback on the same map, and the stock check still passes on the same pk3.

13.4 3.4 Direct light parity

- Lights come from the entity lump, preserved by `-keeplights`, read with the editor's physical model (§15): `light`, `light_spot`, `light_sun` and their keys, with no conversion constant. This replaces sh-renderer's glTF light scale (`kLightIntensityScale`, `src/loader.cpp:18`) and `ComputeLightRadius` (`src/loader.cpp:719`).
- Exit criterion: the editor lighting preview and the engine's direct-light-only render agree from the documented camera, within the tolerance already written down for the benchmark map.

13.5 3.5 Shadow parity and solid hulls

- Load the solid hull mesh that `-shbake` writes into the side file (4.3): one closed convex hull per casting brush, structural and detail, which q3map2 builds from the brush and brushside lumps. Use the hulls as front-face-culled casters; draw surfaces remain receivers. The bake's blockers and the renderer's casters are then one solid set, and nothing is rebuilt at load. No BSP format change: the lumps already keep every plane of every solid brush, including void-facing ones.
- Alpha-tested draw surfaces cast too: surfaces whose shader the editor's caster rule says casts through its alpha (the fence, leaf cards; `trans` with `alphashadow`) are drawn into the shadow maps with their alpha test, alongside the hulls. They are non-solid, so the

hulls alone would drop them, and the editor and sh-baker already cast them.
- Known gap: other brushes with non-solid shaders are dropped from the collision lumps. If such a brush must cast, mark its material to keep it, or emit casters separately.
- Reconcile against the editor's scheme, which deliberately uses *first*-depth casters with normal-offset and slope-scaled bias rather than second depth, because second depth is undefined for patches and models. The constants and their provenance are already tabulated in the gamepack README; that table moves with the pack in Stage 2.
- Exit criterion: the town block's shadows match between editor and engine.

14 Stage 4: SH indirect, baked by q3map2

Outcome. `sh-baker`, inside q3map2, bakes the editor's lights into q3map2's own lightmap pages: SH bands for `renderer_sh`, and the stock lightmap, light grid and vertex colours for the lvlworld build. No BSP format change.

14.1 4.1 Decision: link `sh-baker` into q3map2

Decided. q3map2 links `sh-baker`'s shared library and bakes SH on its own luxels inside `-light` (`-light -shbake`), in place of `-light`'s lighting math. Rejected:

- **Exporter, glTF, xatlas atlas** (the prior pipeline). The bake lives in a separate atlas over a separate, vertex-split mesh; nothing maps it back to the BSP, so `renderer_sh` would have to render a second geometry and keep fog, shader stages, PVS and patches in step with it.
- **Writing xatlas coordinates back into the BSP.** Needs the draw surfaces rewritten to hold split vertices and patches converted to triangles (`-patchmeta`) — the largest option.

Grounds: q3map2 already has the BSP, shader parsing, patch tessellation, image loading, the light entities, lightmap page allocation and light-grid points. The stage passes the editor's light entities to `sh-baker` unchanged, so units agree by construction (§15). Baking in q3map2's lightmap space means `renderer_sh` uses the BSP's own coordinates with no mapping file, and the same bake fills the stock lightmap (revision 6).

14.2 4.2 `sh-baker` changes

Delivered as pull requests to `sh-baker`, reviewed and merged by its maintainer. Each keeps the CLI's current behaviour and its tests passing. There is no separate library boundary: q3map2 includes the headers in `src/` directly and links the `libsh_baker.so` that `sh-baker`'s own CMake builds (4.3), so there is no separate public header.

1. **q3map2 input contract** (B2, openspec change `sh-baker-q3map2-input`, shipped as a stack of seven PRs). *Done (`sh-baker` PRs #54–#60).*
   - An owned material-layer model: the Quake 3 stage stack becomes `sh-baker` types in `material.h`, which the glTF loader and saver translate to and from `SH_material_layers`. No tinygltf in the headers q3map2 includes.
   - A q3map2 loader translation unit (`loader_q3map2.{h,cpp}`) that turns one draw surface into checked tracing geometry: winding flipped to counter-clockwise, malformed input stopped by a `CHECK`, zero normals repaired, degenerate triangles dropped, and tangents from the MikkTSpace code it shares with the glTF loader. Material-less surfaces block light without shading.
   - Baking caller-supplied points: `BakeSHLightMap` bakes any list of points, laid out $N \times 1$, in one call with one BVH. This is how q3map2's luxels are baked (4.3).

This replaces the multi-page UV layouts of earlier drafts. xatlas, as `sh-baker` calls it, never emits several atlases (with `texelsPerUnit` 0 it grows one), `sh-baker`'s rasterizer snaps to whole texels while q3map2 puts lightmap coordinates on luxel centres, and q3map2 already owns the luxel mapping. So `sh-baker` rasterizes nothing for q3map2.

2. **Alpha in shadow rays** (B2b). NEE's visibility test treats any hit as opaque (`light.cpp:265`, `:339`), so fences and leaf cards block direct light completely. The shadow rays honour alpha the way the tracer's indirect rays already do, so they cast through their cut-outs. Kept out of item 1 because it changes bake output; needed by 4.3.
3. **Progress and cancel.** One `BakeSHLightMap` call already builds the Embree device, the BVH and the light trees once for all its points (`baker.cpp:95-96`); it gains progress and cancel callbacks.
4. **Outputs.** The SH side-file format (L0, L1, L2 per page) and the solid hull mesh (3.5), plus an optional Radiance `.hdr` dump of L0 for inspection. `sh-baker` returns SH and irradiance per luxel; q3map2 assembles the pages, the light grid and the vertex colours. No rend2 layout.

The existing light and environment types already cover what q3map2 passes in: point, spot, directional, area lights and a Preetham sky; material-less geometry is already a pure occluder.

14.3 4.3 `-shbake` in q3map2

- **Placement.** `-shbake` runs inside `-light` (`q3map2 -light -shbake`), after `MapRawLightmap` has mapped every raw lightmap's luxels (origins, normals, clusters). It replaces `IlluminateRawLightmap`'s lighting math, and the light-grid and vertex-light passes, with `sh-baker`. `StoreSurfaceLightmaps` then packs the result as usual. It cannot run on the finished BSP: the final pages exist only after `StoreSurfaceLightmaps`, whose packing depends on the lit values.
- **Scene.** Draw surfaces — planar, meta, triangle soups and tessellated patches — go through `sh-baker`'s q3map2 loader as receivers. Materials from q3map2's shader info: the base-colour image, the `qer_pbr_*` maps and factors (q3map2's shader parser learns them), additive stages as emitters. Lights are the editor's `light`, `light_spot` and `light_sun` entities read with the physical model of §15, not q3map2's light parsing. Occluders follow the editor's shadow-caster rule (§2).
- **Solid hulls.** q3map2's draw surfaces are thin, and rays can slip through the cracks between them. The blockers are one closed hull per casting brush, structural and detail, built from the BSP's brush lumps with q3map2's winding code (`BaseWindingForPlane`, `ChopWindingInPlace`). Each plane is pushed inward by ε, a little more than the sensor's `tnear`, so the visible face is always hit first; bevel sides are skipped; each hull is added as a material-less surface, which blocks light without shading. The draw surfaces stay the receivers and own the luxels. Brushless surfaces (patches, `misc_model` props, foliage) stay thin. The un-shrunk hulls go into the side file for 3.5.
- **Bake and write.** One `sh-baker` call bakes every mapped luxel as a caller-supplied point, with one BVH for the map. The SH side data must follow `StoreSurfaceLightmaps`' approximation to vertex light, solid stamps and deduplication, or those must be disabled when baking SH. All bands to the side file in `maps/<name>/`, inside the level's single pk3. Never `lm_*.hdr`, which stock rend2 would load. Always also write direct plus indirect irradiance into the BSP's 8-bit lightmap lump, the light grid (ambient, directed, direction from SH at the grid points) and vertex colours, so the lvlworld build carries the same light `renderer_sh` shows. The irradiance-to-byte encoding is chosen here (§15).
- **Anti-aliasing.** From `-super N`: raw lightmaps are mapped at $N\times$ resolution, each super-luxel is baked as a point, and `StoreSurfaceLightmaps` averages them; SH coefficients are linear, so the side data averages the same way. `-samples`' adaptive subsampling is part of

the replaced lighting and is not available.

- **Units.** The sensor's ray offset (`tnear`, 0.005) is in metres, while q3map2 works in Quake units, so the stage converts units or scales the scene.
- **Gaps the stage reports.** q3map2-only light inputs — `spawnflags`, `fade`, `q3map_sun`, `q3map_skylight`, `q3map_surfacelight` — are not read; the stage warns on them.
- **Build.** `sh-baker` is the `tools/sh-baker` submodule, built by its own CMake in Release (never Debug, which adds `-fsanitize=address`) into `libsh_baker.so`; its dependencies (glog, Embree, TBB, xatlas) load through the library. Under an opt-in flag (`SH_BAKER=1`) q3map2's `Makefile` adds `tools/sh-baker/src` and Eigen to the include path, links `-lsh_baker` (and `-lembree4` if the stage calls Embree itself), and sets an rpath or installs the library beside q3map2; a default q3map2 build needs none of this. The headers the stage includes (`loader_q3map2.h`, `baker.h`) need only `src` and Eigen, because `sh-baker` declares the Embree handles itself (4.2 item 1). The stage lives in a new `light_sh.cpp`, the only q3map2 file that includes `sh-baker`'s headers; they compile cleanly under q3map2's `-fno-exceptions -fno-rtti`. That file alone is compiled with `-march=native`, because `sh-baker`'s Release build always uses it and it changes Eigen's alignment: `Geometry` (through its `Affine3f` transform) is 32-byte aligned with AVX and 16 without, and a mismatch lets the library run aligned AVX loads on objects q3map2 allocated. `sh-baker` logs through glog to stderr, not through `Sys_Printf`, so the editor's build monitor does not show those lines.
- **Licence.** `sh-baker` is MIT, compatible with q3map2's GPL-2.0-or-later. Embree and TBB are Apache-2.0, compatible with GPLv3 but not GPLv2, so a q3map2 binary built with `SH_BAKER=1` is distributed under GPLv3, which "or any later version" permits.

14.4 4.4 Runtime and exit

- `renderer_sh` reads the side file and runs the indirect BRDF: Ramamoorthi–Hanrahan cosine convolution for diffuse, SH evaluated along the reflection vector for specular. A map without the side file falls back to the BSP lightmap.
- Exit criterion: the town block passes the stock check, and `renderer_sh` renders all three bands.
- Editor preview of baked SH stays optional and last. The editor has no lightmap coordinates on source geometry, and the engine covers the need.

15 Units and colour space (cross-cutting)

Decided (revision 6): the editor's physical light model is the one light model. The editor preview, `sh-baker` (through `-shbake`) and `renderer_sh` read the same entities the same way. Nothing reproduces q3map2's `-light`, and there is no conversion constant between consumers.

- **Point.** `light`: `intensity` is radiant flux (default 50000); radiant intensity is `intensity`$/4\pi$, falling off with the inverse square of the distance in raw map units; culled beyond `radius` (default 256). Never casts shadows.
- **Spot.** `light_spot`: the same flux concentrated in the cone, radiant intensity `intensity`$/(2\pi(1-\cos$ `cone`$))$; `cone` is the outer half-angle (default 45°), `cone_inner` the inner one (default 30°), smoothstep between; direction from `target`, else `angles`; culled beyond `radius`.
- **Sun.** `light_sun`: `intensity` is the irradiance on a surface facing it (default 3), direction from `angles`, no cutoff. One sun is honoured, the first in map order. `q3map_sun` is not read.

- **Colour and shadows.** `_color` is linear RGB. Spots and suns cast unless `_shadows 0`.
- **Lightmap encoding.** `-shbake` converts irradiance to the 8-bit lightmap lump. The stock renderers at default cvars in a window shift lightmap bytes by 2, so a byte saturates at 64; the encoding is chosen to look good there, for example a roll-off toward that limit rather than a hard clip. It is recorded with the stage and judged by 0.3.
- **The placeholder.** q3map2's own reading of light entities (its ×7500 intensity, distance clamp and envelope) affects only the placeholder lightmap before 4.3 and is not matched.

Consumer	Current scale	Where
`sh-renderer`	×1/200 on the glTF intensity; replaced by reading the entities (3.4)	`sh-renderer/src/loader.cpp:18`, `kLightIntensityScale`
Editor	flux in the tens of thousands, raw map units, no metre conversion; this is the model	`setup/data/gamepacks/pbr/README.md`
`sh-baker`	physical intensity from its glTF loader	set by `-shbake` from the entities, unchanged

One spatial unit is fixed for all consumers: raw map units, as the editor and q3map2 use.

16 World-building conventions for the town

Authoring rules, not code, but they determine whether the visibility system works and whether the reference look is reachable.

- **Low sky ceiling just above the camera.** VIS treats sky as opaque, so the town becomes a network of corridors walled by buildings and the PVS behaves like an indoor level.
- **Fog colour equals sky colour.** Buildings end at the ceiling; with matched colours both fade to the same value and the seam disappears. Far plane at fog opacity distance.
- **Building shells are structural brushes.** Only structural brushes block VIS. Ornament is detail. Props, pipes and clutter are models.
- **Hint brushes across long streets** at intervals so far clusters drop out; area portals in doors between street and interiors.
- **Clamp camera pitch** so the ceiling is never inspected from below.
- **Set worldspawn `_blocksize`** to zero or a large value for the flat town, to avoid pointless axial splits.
- **Silhouette over detail.** At reference fog density, only near-field surfaces are legible; budget texture and geometry effort into the first few metres and into readable silhouettes at the fog limit.
- **Author for a moving light.** Surfaces are seen by a flashlight cone at grazing angles, which is the worst case for normal maps and the best case for them to matter. Judge materials under the carried light, not under the sun.
- **Lightmapped surfaces only**, so every surface receives SH and baked light (4.3).
- **Vegetation is lightmapped `misc_model` meshes.** Model triangles are vertex-lit by default, so vegetation shaders carry `q3map_forceMeta`. Grass is static crossed quads, not `autosprite`, which only the engine would turn to the camera.
- **Use a spot where occlusion matters.** Point lights do not cast shadows in the editor preview; a `light_spot` does.
- **Stock content only.** Every entity, key and shader keyword obeys §6 and §7; the stock check is the arbiter.

17 Risks

Risk	Mitigation
Stock rend2 picks up PBR data by accident	The contract: reserved suffixes, no rend2 keywords, no `lm_*.hdr`. The stock check runs the shipped pk3.
The fork's renderers drift from upstream	Accepted: they are machine-converted to C++ and otherwise original. Each stock-check run records the fork's commit.
The lvlworld build looks flat next to `renderer_sh`	Expected: no normal maps, no specular. Stage 4 writes baked direct and indirect light into the lightmap lump with an encoding tuned for vanilla; 0.3 measures it.
Enhanced-only factors make the stock look diverge	Bake factors into textures where the stock look matters.
The placeholder lightmap differs from the preview until 4.3	Accepted: before 4.3 the stock check gates only on loading and warnings.
`light_spot`/`light_sun` print a line in the stock game	Harmless (the entity is discarded); allowlisted in the stock check.
The overlay overwrites the downloaded `Q3.game`	The overlay is tracked in `setup/data/gamepacks/pbr/`; rerun its install after re-downloading the gamepacks.
`sh-baker` changes disturb its CLI	Each PR keeps the CLI's behaviour and tests; the API is reviewed before q3map2 uses it.
q3map2's lightmap pages are a worse layout than xatlas's	Density is controlled per shader (`q3map_lightmapSampleSize`), globally (`-samplesize`) and by page size (`-lightmapsize`, `-external`). Accepted in exchange for a bake that maps to the BSP.
Embree and TBB complicate q3map2's build	Opt-in `SH_BAKER=1`; the default build is unchanged.
The `renderer_sh` feature set is open-ended	The accepted cost of the §3.1 decision. Bounded by the tiering in §3.3: Tier 1 is walkable, Tier 3 is stubbed indefinitely. `opengl2` remains the fallback for anything unimplemented, so no gap blocks play.
The `ri.*` refactor spreads through sh-renderer	The one non-additive part of §3.1; done first, before feature work, so nothing is ported twice.
Point lights over-light occluded areas in the preview	Authoring convention: spots where occlusion matters.
Texture loader preferences alter colour data	Audit gamma and compression paths before choosing where linearisation happens.
Non-solid brushes needed as casters are dropped from the collision lumps	Material key to keep them, or a small editor-side caster export.
Long straight streets defeat the PVS	Fog far plane plus hint brushes; accept that the fog is the real culler there.
Qt GL context lacks needed features for FBO and half-float	Explicit 3.3 compatibility profile in `QSurfaceFormat`; the legacy code keeps working.

18 Dependency summary

Item	Depends on	Note
0.1 Convert bench map	—	Unblocked; also the parity asset for everything below.
0.2 First town block	0.1	Unblocked, under `Q3.game`, with `-keeplights`.
0.3 Measure stock fallback	0.2, 4.3	Zero code. Judges `sh-baker`'s lightmap on the lvlworld build.
0.4 Stock check	0.1	Fork build plus `tools/stockcheck/`; gates every later stage.
1 Silent Hill look	0.2, 0.3	Content and cvars; the flashlight needs the game-code decision.
2.2 steps 1–4	—	Additive; materials, preview and light entities under `Q3.game`.
2.2 step 5	0.1	Deletes `pbr.game`; keeps the `pbr` entity module.
3.1 `ri.*` refactor	—	Decided; not additive, so it goes first.
3.3 Tier 1 features	3.1, 0.2	Walkable town under `cl_renderer sh`.
3.3 Tier 2 features	Tier 1	Sky and fog first; they define the town.
3.4 Direct light parity	2.2, Tier 1	Editor and engine read lights identically.
3.5 Shadow parity	3.4, 4.3	Hulls loaded from the side file, validated against the town block.
4.2 `sh-baker` PRs	—	Can start now; reviewed and merged one at a time. Item 1 ships as a stack of seven PRs.
4.3 `-shbake`	4.2, 0.2, 0.4	Runs inside `-light` on q3map2's own luxels; bakes SH and the stock lightmap, grid and vertex colours, and writes the hull mesh to the side file; stock check on the single pk3.
4.4 SH at runtime	4.3, Tier 1	All bands from the side file.
4.x Editor SH preview	4.4	Optional.